As part of our hiring process, we'd like you to complete a short design exercise. The full problem statement is attached to the email.

A few quick notes before you dive in:

• Complete the assignment in 6 to 8 hours and revert to recruiter
• Use any AI tools you would use on the job
• **Submit via a public GitHub repository link.**
• Submission deadline: within 36 hours from the receipt of the email.

After we receive your submission, we'll review and score the exercise and reach out to you for next steps.

Epic: Practical FDE Assessment — Idempotent Background Transaction Processing Service

1. Epic Overview

Epic Name: Idempotent Background Transaction Processing Service
Assessment Duration: 2 hours
Assessment Type: AI-assisted practical engineering assessment
Business Context: Modern enterprise platforms frequently receive transaction requests asynchronously from upstream systems, APIs, files, schedulers, queues, or retrying clients. These requests may arrive out of order, may be duplicated, and may be retried after partial failures. The candidate must build a background processing service that processes transaction requests safely, consistently, and with sufficient operational resilience.
Epic Goal: Assess whether the candidate can design and implement a practical, production-aware background service that processes transaction requests idempotently, handles duplicates and out-of-order delivery, applies essential non-functional requirements, and demonstrates the FDE mindset of rapid problem understanding, clean implementation, AI-assisted execution, and outcome ownership.

2. FDE Readiness Intent

This assessment is not intended to be a pure coding challenge or a feature-completion exercise. It is intended to validate whether the candidate demonstrates the traits expected from a Forward Deployed Engineer.

For this assessment, candidate is expected to show:

- Strong core engineering and architecture fundamentals.
- Ability to translate an ambiguous business problem into a working technical solution.
- Ability to use AI-assisted development effectively without blindly accepting generated output.
- Practical understanding of APIs, background jobs, distributed processing, retries, failure handling, observability, and enterprise-grade quality.
- Ability to reason about customer impact, operational safety, and production behavior.
- Clear communication of design choices, trade-offs, assumptions, and next steps.
- Ownership mindset: solve the right problem, not just write code.

FDE Level	Assessment Positioning	Expected Signal

FDE L1	Customer-aware engineer	Can implement a guided, scoped solution with AI assistance, basic idempotency, tests, and clear explanation.
FDE L2	Customer-facing engineer	Can independently design and implement a more complete solution with resilience, ordering strategy, observability, edge-case handling, and production-readiness trade-offs.

4. Common Problem Theme

Both levels use the same business theme:

Build an idempotent background transaction processing service that can process transaction requests arriving in any order, including duplicate requests, without causing duplicate processing or inconsistent transaction state.

The service should simulate a real enterprise background job. It may read from an in-memory list, local file, test input, mocked queue, database table, or simple API endpoint, depending on the candidate's chosen technology stack and time available.

The core challenge is not the input mechanism. The core challenge is safe transaction processing under duplicate and out-of-order request conditions.

5. Common Transaction Model

Candidates may define their own model, but the minimum expected transaction request should include:

Field	Description
transactionId	Unique business transaction identifier. Used for idempotency.
accountId	Account or customer reference.
transactionType	CREDIT, DEBIT, REVERSAL, or equivalent.
amount	Transaction amount.
currency	Currency code.
sequenceNumber or eventTime	Used to reason about ordering where applicable.
requestId or idempotencyKey	Unique request identifier for retry and duplicate detection.
status	RECEIVED, PROCESSING, PROCESSED, DUPLICATE, FAILED, RETRY_PENDING, or equivalent.

6. Common Non-Functional Requirements

The candidate should consider the following NFRs. The expected depth differs between FDE L1 and FDE L2.

6.1 Idempotency

- Duplicate transaction requests must not result in duplicate balance updates, duplicate ledger entries, or duplicate downstream side effects.
- The same request processed multiple times should produce the same final business outcome.

- The service should persist or simulate a processed-request store, transaction state store, or deduplication mechanism.

6.2 Ordering

- The service must tolerate requests arriving in any order.
- The candidate should define how out-of-order requests are handled.
- Acceptable strategies include buffering, sequence validation, state transition rules, pending state, or deterministic conflict handling.

6.3 Resilience

- The service should handle transient processing failures.
- The service should support retry behavior without breaking idempotency.
- Failed records should be traceable and recoverable.

6.4 Data Consistency

- Processing should not leave partially updated transaction state.
- Candidate should show awareness of transaction boundaries, atomic updates, or compensating logic.

6.5 Observability

- The service should log key processing events.
- It should be possible to identify processed, duplicate, failed, and retried transactions.
- FDE L2 should include better operational visibility such as summary metrics, structured logs, or processing report.

6.6 Security and Data Handling

- Do not log sensitive customer data unnecessarily.
- Validate input before processing.
- Avoid hardcoded secrets.
- Use safe error handling.

6.7 Maintainability

- Code should be modular and readable.
- Business rules should be separated from infrastructure concerns where practical.
- Tests should cover key idempotency and failure scenarios.

7. Assessment Ground Rules

- Candidate may use AI tools such as GitHub Copilot or equivalent.
- Candidate must be able to explain the generated code and design choices.
- Candidate may choose Java, .NET, or an equivalent stack.
- No external paid services are required.
- No complex UI is required.
- No production database is mandatory; in-memory persistence or lightweight local persistence is acceptable if clearly explained.

- Focus should remain on the background processing logic and FDE readiness signals.
- Completion of every feature is not mandatory. Clear design, prioritization, trade-off explanation, and working core behavior are more important.

9. L2 Practical Assessment Requirement

9.1 Epic Statement

As an FDE L2 candidate, I need to design and build a more production-aware background transaction processing service that can handle duplicate, out-of-order, and retry-prone transaction requests with resilience and observability, so that I can demonstrate independent solution ownership and customer-facing FDE readiness.

9.2 Problem Statement

Build a background transaction processing service that receives transaction requests that may arrive in any order and may include duplicates. The service must process valid transactions idempotently, detect duplicates, handle retryable failures, and maintain clear transaction state.

The candidate should design the service as if it could later be connected to a real queue, scheduler, API, or database-backed job processor. The implementation may remain lightweight for the two-hour assessment, but the design should show production-aware thinking.

The service should support:

- Idempotent processing.
- Duplicate detection.
- Out-of-order request handling strategy.
- Retry-safe failure handling.
- Clear transaction status lifecycle.
- Processing summary and basic operational visibility.
- Tests covering important edge cases.

9.3 Scope

In Scope

- Transaction request ingestion from a simple source.
- Background processing component.
- Idempotency store or equivalent state mechanism.
- Transaction state lifecycle.
- Duplicate and replay handling.
- Out-of-order request strategy.
- Retry handling for simulated transient failures.
- Failed transaction handling.
- Logging or processing report.
- Unit tests for key scenarios.
- Design explanation and production-readiness notes.

Out of Scope

- Full distributed transaction implementation.
- Real enterprise queue integration unless the candidate chooses to add a simple adapter.
- Multi-region deployment.

- Complex UI.
- Full security implementation.
- Full observability stack.

9.4 Functional Requirements

ID	Requirement
L2-FR-01	The service shall accept transaction requests from a configurable or replaceable input source.
L2-FR-02	The service shall validate mandatory fields and reject malformed records.
L2-FR-03	The service shall maintain transaction processing state.
L2-FR-04	The service shall support a status lifecycle such as RECEIVED, PROCESSING, PROCESSED, DUPLICATE, FAILED, and RETRY_PENDING.
L2-FR-05	The service shall process each unique business transaction only once.
L2-FR-06	The service shall detect duplicate requests even if they arrive later in the batch.
L2-FR-07	The service shall define and implement a strategy for out-of-order events.
L2-FR-08	The service shall simulate at least one retryable failure scenario.
L2-FR-09	The service shall retry failed transient records without creating duplicate effects.
L2-FR-10	The service shall separate business processing logic from input handling where practical.
L2-FR-11	The service shall produce a processing summary with counts by status.
L2-FR-12	The service shall include tests for duplicate, out-of-order, invalid, and retry scenarios.

9.5 Out-of-Order Handling Expectations
The candidate may choose one of the following approaches and explain the trade-off:
Option A: Sequence-Based Processing

- Each transaction request has a sequence number.
- Requests with future sequence numbers are marked pending.
- Requests are processed when prior required sequence numbers are available.
- Missing sequences are reported.

Option B: State-Based Processing

- Each transaction has a valid state transition model.
- Events are processed only if the transition is valid.
- Invalid or premature transitions are marked pending or rejected.

Option C: Deterministic Latest-State Processing

- The service uses eventTime or version to determine the latest valid state.
- Older duplicates or stale updates are ignored.
- Candidate clearly explains why this is acceptable for the selected transaction domain.

For a two-hour assessment, a simple and well-explained approach is acceptable. The candidate is not expected to build a full event-sourcing platform.

9.6 L2 Resilience Expectations

The candidate should implement or clearly simulate:

- Transient failure for one or more records.
- Retry behavior with a maximum retry count.
- No duplicate transaction effect after retry.
- Failed state after retry exhaustion.
- Clear logging or result reporting.

9.7 L2 Acceptance Criteria

- Duplicate requests do not result in duplicate side effects.
- Out-of-order requests are handled through a defined strategy.
- Retry behavior is safe and does not violate idempotency.
- Invalid records are isolated and do not stop the full batch.
- The service shows a clear transaction status lifecycle.
- Candidate can explain production implications such as persistence, concurrency, queue semantics, monitoring, and recovery.
- Candidate can explain how AI assistance improved speed and how generated code was validated.
- Candidate can identify trade-offs and next steps for production hardening.

9.8 L2 Suggested Test Data

Input Order	**transactionId**	**requestId**	**sequence**	**accountId**	**type**	**amount**	**Expected Outcome**
1	TXN-2002	REQ-2	2	ACC-201	DEBIT	40	Pending or processed based on chosen strategy
2	TXN-2001	REQ-1	1	ACC-201	CREDIT	100	Processed

3	TXN-2002	REQ-2-DUP	2	ACC-201	DEBIT	40	Duplicate if already processed
4	TXN-2003	REQ-3	3	ACC-201	CREDIT	25	Processed
5	TXN-2004	REQ-4	4	ACC-202	DEBIT	-10	Failed validation
6	TXN-2005	REQ-5	5	ACC-203	CREDIT	500	Retry once, then processed or failed based on simulation

9.9 L2 Expected Deliverables

- Source code.
- Execution instructions.
- Sample input and output.
- Unit tests or test evidence.
- Design note covering:
    - Architecture.
    - Idempotency design.
    - Duplicate detection logic.
    - Out-of-order handling strategy.
    - Retry and failure handling.
    - Observability approach.
    - AI-assisted development usage.
    - Production hardening recommendations.